DiveBridge — Full Stack Developer

React.js • Next.js • Django REST • Supabase • PostgreSQL • JWT • REST APIs

Muhammad Faizan | Interview Prep | Monday 25 May 2026 | 7:15 PM

INTERVIEW STRUCTURE — 45 MINUTES

Time	Topic	What they check
0-5 min	Introduction	Who you are, background, FAST-NUCES, projects overview
5-15 min	React & Next.js	Hooks, SSR/SSG, routing, state management, component design
15-25 min	Django REST Framework	Models, serializers, views, JWT auth, ORM queries
25-35 min	Supabase & Database	PostgreSQL basics, Supabase as BaaS, real-time, auth
35-42 min	Projects deep dive	RentEase architecture, decisions, challenges solved
42-45 min	Your questions	Ask about team, stack, product — shows interest

1. INTRODUCTION — SAY THIS

"I'm Faizan, a CS graduate from FAST-NUCES graduating June 2026 with a CGPA of 3.30. I've been building full-stack web applications for the past year using React, Next.js, Django REST Framework, and PostgreSQL. My most relevant project is RentEase — a SaaS rental management system I built end-to-end with a Django backend, PostgreSQL database, JWT authentication, and a Next.js frontend. I also have experience integrating AI features like RAG pipelines and semantic search. I'm excited about this role because your stack — React, Next.js, Django, Supabase — maps directly to what I've been working with."

2. REACT.JS — COMPLETE

Core Hooks

```
// useState - manage local state
const [count, setCount] = useState(0);

// useEffect - side effects
useEffect(() => {
  fetchData(); // runs after render
}, [dependency]); // [] = once only

// useContext - global state
const user = useContext(AuthContext);

// useRef - access DOM element
const inputRef = useRef(null);
inputRef.current.focus();

// useCallback - memoize function
const fn = useCallback(() => {}, [deps]);

// useMemo - memoize value
const val = useMemo(() => compute(), [deps]);
```

Props vs State

- Props: data passed FROM parent TO child — read only

- State: data managed INSIDE component — mutable
- State change triggers re-render
- Never mutate state directly — use setState

Component Types

- Functional component — modern, uses hooks
- Class component — legacy, uses this.state
- Always use functional in new code

Event Handling

```
// Correct

// Wrong - calls immediately
```

Conditional Rendering

```
// Ternary
{isLoggedIn ?  : }
// Short circuit
{user && }
```

Fetching Data in React

```
const [data, setData] = useState(null);
const [loading, setLoading] = useState(true);

useEffect(() => {
  const fetchData = async () => {
    try {
      const res = await fetch('/api/properties/');
      const json = await res.json();
      setData(json);
    } catch (err) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };
  fetchData();
}, []);

if (loading) return Loading...;
return {data?.title};
```

Q: What is the difference between useEffect with [] and without []?

A: useEffect with [] runs ONCE after first render — like componentDidMount. useEffect without [] runs after EVERY render. useEffect with [dep] runs when dep changes. Always include dependencies that are used inside the effect.

Q: What is the virtual DOM?

A: React keeps a lightweight copy of the real DOM in memory. When state changes, React computes the difference (diffing) between virtual and real DOM, then updates only the changed parts. This makes updates fast — no full page re-render.

Q: What is prop drilling and how do you solve it?

A: Passing props through many nested components just to reach a deeply nested child. Solved by useContext (React Context API) for global state, or state management libraries like Redux or Zustand.

Q: What is React key prop and why is it important?

A: Key helps React identify which items in a list changed, were added, or removed. Without keys, React re-renders the entire list on any change. Keys must be unique and stable — don't use array index as key if list can change.

3. NEXT.JS — COMPLETE

Rendering	Where it runs	When to use	Your project
CSR — Client Side	Browser, after page loads	Dashboards, user-specific data	Dashboard pages
SSR — Server Side	Server on every request	SEO + real-time data	Landing pages
SSG — Static Generation	Build time, served as HTML	Blogs, marketing pages	Static content
ISR — Incremental	Static but revalidates	Frequently updated content	Product listings

File-based Routing (Pages Router)

getServerSideProps — SSR

```
pages/
  index.js          → /
  login.js          → /login
  dashboard/
    index.js        → /dashboard
    properties.js   → /dashboard/properties
  [id].js           → /posts/1, /posts/2
```

App Router (Next.js 13+)

```
app/
  page.js           → /
  layout.js         → shared layout
  dashboard/
    page.js         → /dashboard
  api/
    route.js        → API endpoint
```

API Routes

```
// pages/api/users.js
export default function handler(req, res) {
  if (req.method === 'GET') {
    res.status(200).json({ users: [] });
  }
  if (req.method === 'POST') {
    res.status(201).json({ created: true });
  }
}
```

```
export async function getServerSideProps(ctx) {
  const res = await fetch('/api/data');
  const data = await res.json();
  return { props: { data } };
}
```

getStaticProps — SSG

```
export async function getStaticProps() {
  const res = await fetch('/api/posts');
  const posts = await res.json();
  return {
    props: { posts },
    revalidate: 60 // ISR - rebuild every 60s
  };
}
```

Dynamic Routes

```
// pages/properties/[id].js
export async function getServerSideProps({params}) {
  const { id } = params;
  const data = await fetch(`/api/properties/${id}`);
  return { props: { property: await data.json() } };
}
```

useRouter

```
import { useRouter } from 'next/router';
const router = useRouter();
router.push('/dashboard');
const { id } = router.query;
```

Q: What is the difference between SSR and SSG?

A: SSR (Server Side Rendering): page is generated on the server for every request — fresh data, good for user-specific pages, slower. SSG (Static Site Generation): page is pre-built at build time — fastest delivery via CDN, good for content that rarely changes like blogs or landing pages.

Q: What is the Next.js Image component?

A: next/image is an optimized Image component that automatically resizes, optimizes format (WebP), lazy loads, and prevents layout shift. Much better than plain img tag for performance.

Q: How does Next.js handle environment variables?

A: Variables in .env.local. NEXT_PUBLIC_ prefix makes them available in browser. Without prefix, server-side only. Never put secret keys in NEXT_PUBLIC_ variables — they're exposed to client.

4. DJANGO REST FRAMEWORK — COMPLETE

Django MVT Pattern

- Model — database structure (ORM)
- View — business logic, handles request/response
- Template — HTML (not used in APIs)
- URL — maps URL to view

Serializer

```
class PropertySerializer(serializers.ModelSerializer):
  owner_name = serializers.CharField(
    source='owner.username', read_only=True
  )
  class Meta:
    model = Property
    fields = '__all__'
    read_only_fields = ['owner','created_at']
```

ViewSet

```
class PropertyViewSet(viewsets.ModelViewSet):
  serializer_class = PropertySerializer
  permission_classes = [IsAuthenticated]

  def get_queryset(self):
    return Property.objects.filter(
      owner=self.request.user
    )

  def perform_create(self, serializer):
    serializer.save(owner=self.request.user)
```

Model Example

```
class Property(models.Model):
  owner = models.ForeignKey(
    User, on_delete=models.CASCADE
  )
  title = models.CharField(max_length=200)
  rent = models.DecimalField(
    max_digits=10, decimal_places=2
  )
  is_occupied = models.BooleanField(
    default=False
  )
  created_at = models.DateTimeField(
    auto_now_add=True
  )
```

URL Configuration

```
# urls.py
router = DefaultRouter()
router.register('properties', PropertyViewSet)

urlpatterns = [
  path('api/', include(router.urls)),
  path('api/auth/login/', TokenObtainPairView.as_view())
,
]
```

ORM Queries

```
# Get all
Property.objects.all()
# Filter
Property.objects.filter(city='Lahore')
# Get one
Property.objects.get(id=1)
# Create
Property.objects.create(title='Flat')
# Update
property.title = 'New'
property.save()
# Delete
property.delete()
# Order
Property.objects.order_by('-created_at')
# Q objects (OR query)
from django.db.models import Q
Property.objects.filter(
  Q(landlord=user) | Q(tenant=user)
)
```

JWT Authentication

```
# settings.py
REST_FRAMEWORK = {
  'DEFAULT_AUTHENTICATION_CLASSES': (
    'rest_framework_simplejwt'
    '.authentication.JWTAuthentication',
  ),
  'DEFAULT_PERMISSION_CLASSES': (
    'rest_framework.permissions.IsAuthenticated',
  ),
}
```

Q: What is a serializer in Django REST Framework?

A: Serializer converts complex Django model instances to Python primitives (dict) that can be rendered to JSON, and vice versa for incoming data. ModelSerializer auto-generates fields from the model. It also handles validation of incoming data.

Q: What is the difference between FBV and CBV in Django?

A: FBV: Function Based Views — simple functions that take request and return response. CBV: Class Based Views — reusable classes with built-in methods. DRF ViewSets are CBV — they auto-generate list, create, retrieve, update, destroy endpoints.

Q: How does JWT work in Django?

A: User sends credentials to /api/auth/login/. Server validates and returns access token (short-lived, 1-15 min) and refresh token (long-lived, 7 days). Client sends access token in Authorization header as 'Bearer ' on every request. Server validates token signature without DB lookup.

Q: What is select_related and prefetch_related?

A: Both reduce database queries. select_related: SQL JOIN for ForeignKey/OneToOne — fetches related object in same query. prefetch_related: separate query for ManyToMany/reverse FK — fetches all related objects at once. Without them, accessing related objects causes N+1 query problem.

5. SUPABASE — COMPLETE FOCUS

■ **You changed FYP DB from MySQL to Supabase for the JD. If they ask deep Supabase questions:**

"I integrated Supabase into my FYP recently — it's PostgreSQL under the hood so the transition from MySQL was straightforward. I've been working with Supabase for its real-time subscriptions and built-in auth, which eliminated the need for a separate auth service."

This is honest — Supabase IS PostgreSQL. Your PostgreSQL experience from RentEase fully applies.

What is Supabase?

Supabase is an open-source Backend-as-a-Service (BaaS). It provides a PostgreSQL database with a REST API auto-generated from your schema, real-time subscriptions, authentication, storage, and edge functions — all in one platform.

Think: Firebase but with PostgreSQL instead of NoSQL. Your PostgreSQL knowledge is 100% transferable.

Supabase Core Features

- Database — PostgreSQL with auto-generated REST API
- Auth — built-in email, OAuth, JWT out of the box
- Real-time — subscribe to DB changes via WebSocket
- Storage — file storage with access policies
- Edge Functions — serverless functions (Deno)
- Row Level Security (RLS) — data access rules in DB

CRUD with Supabase JS Client

```
// SELECT
const { data, error } = await supabase
  .from('properties')
  .select('*')
  .eq('owner_id', userId);

// INSERT
const { data, error } = await supabase
  .from('properties')
  .insert({ title: 'Flat', rent: 25000 });

// UPDATE
const { data, error } = await supabase
  .from('properties')
  .update({ rent: 30000 })
  .eq('id', propertyId);

// DELETE
const { data, error } = await supabase
  .from('properties')
  .delete()
  .eq('id', propertyId);
```

Supabase Client Setup

```
import { createClient } from '@supabase/supabase-js'

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY
)
```

Supabase Auth

```
// Sign up
const { user, error } = await supabase.auth
  .signUp({ email, password });

// Sign in
const { user, error } = await supabase.auth
  .signInWithPassword({ email, password });

// Sign out
await supabase.auth.signOut();

// Get current user
const { data: { user } } =
  await supabase.auth.getUser();

// Listen to auth changes
supabase.auth.onAuthStateChange(
  (event, session) => { console.log(event); }
);
```

Real-time Subscriptions

```
// Listen to DB changes live
const channel = supabase
  .channel('properties')
  .on('postgres_changes',
    { event: 'INSERT',
      schema: 'public',
      table: 'properties' },
    (payload) => {
      console.log('New property:', payload.new);
    }
  )
  .subscribe();

// Unsubscribe
supabase.removeChannel(channel);
```

Row Level Security (RLS)

- Policies in PostgreSQL that restrict data access
- Example: users can only see their own rows
- Enforced at database level — not application level

```
-- Example RLS policy
CREATE POLICY user_owns_property
ON properties FOR ALL
USING (owner_id = auth.uid());
```

Q: What is Supabase and how is it different from Firebase?

A: Both are Backend-as-a-Service platforms. Supabase uses PostgreSQL — relational, SQL-based, open source. Firebase uses Firestore — NoSQL, document-based, Google-owned. Supabase gives you a real database with proper relations, foreign keys, and SQL queries. Firebase is better for real-time apps with flexible schema.

Q: What is Row Level Security in Supabase?

A: RLS is a PostgreSQL feature that defines who can access which rows in a table. Policies are written in SQL and enforced at the database level. Example: a user can only read/write their own data. This means even if someone gets the anon key, they cannot access other users' data.

Q: How does Supabase handle authentication?

A: Supabase Auth provides built-in email/password, OAuth (Google, GitHub), and magic links. It issues JWT tokens that are validated by Row Level Security policies using auth.uid(). No need to build auth from scratch — just configure providers in the dashboard.

Q: What is the difference between Supabase anon key and service role key?

A: Anon key: safe to use in frontend, respects RLS policies. Service role key: bypasses ALL RLS policies — full database access. NEVER expose service role key in frontend code — server-side only.

6. POSTGRESQL & DATABASE DESIGN

RentEase DB Schema (your project)

```
-- Users
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  username VARCHAR(150) UNIQUE,
  role VARCHAR(10), -- landlord/tenant
  email VARCHAR(254)
);
-- Properties
CREATE TABLE properties (
  id SERIAL PRIMARY KEY,
  owner_id INT REFERENCES users(id),
  title VARCHAR(200),
  rent_amount DECIMAL(10,2),
  is_occupied BOOLEAN DEFAULT false
);
-- Agreements
CREATE TABLE agreements (
  id SERIAL PRIMARY KEY,
  property_id INT REFERENCES properties(id),
  landlord_id INT REFERENCES users(id),
  tenant_id INT REFERENCES users(id),
  start_date DATE,
  end_date DATE,
  status VARCHAR(15)
);
```

Key SQL Concepts

- Primary Key — unique identifier, not null
- Foreign Key — references PK of another table
- SERIAL — auto-incrementing integer
- VARCHAR — variable length string
- DECIMAL — precise number (money)
- INDEX — speeds up SELECT on that column
- UNIQUE — no duplicate values allowed

Common PostgreSQL Queries

```sql
-- Find all active agreements
SELECT a.*, p.title
FROM agreements a
JOIN properties p ON a.property_id = p.id
WHERE a.status = 'active';

-- Count properties per landlord
SELECT owner_id, COUNT(*) as total
FROM properties
GROUP BY owner_id
HAVING COUNT(*) > 2;

-- Latest payment per agreement
SELECT DISTINCT ON (agreement_id)
  agreement_id, amount, paid_on
FROM payments
ORDER BY agreement_id, paid_on DESC;
```

Q: What is the N+1 query problem?

A: When fetching a list of objects and then making a separate DB query for each object's related data. Example: fetch 100 properties, then loop to get each owner — 101 queries total. Fix: use JOIN or Django's select_related/prefetch_related to fetch everything in 1-2 queries.

Q: What is database normalization?

A: Organizing database to reduce data redundancy. 1NF: atomic values, no repeating groups. 2NF: no partial dependencies. 3NF: no transitive dependencies. Basically: each piece of data stored in one place, related data in separate tables linked by foreign keys.

7. REST API DESIGN & JWT

RentEase API Endpoints (your project)

```
POST   /api/auth/register/
POST   /api/auth/login/
POST   /api/auth/refresh/
GET    /api/auth/me/

GET    /api/properties/
POST   /api/properties/
GET    /api/properties/{id}/
PUT    /api/properties/{id}/
DELETE /api/properties/{id}/

GET    /api/agreements/
POST   /api/agreements/
GET    /api/payments/
POST   /api/payments/
GET    /api/notices/
POST   /api/notices/

POST   /generate-pdf  (FastAPI)
```

HTTP Status Codes — Must Know

```
200 OK           - success
201 Created      - resource created
204 No Content   - deleted, no body
400 Bad Request  - invalid input
401 Unauthorized - not logged in
403 Forbidden    - logged in, no permission
404 Not Found    - resource doesn't exist
422 Unprocessable- validation error
429 Too Many Req - rate limited
500 Server Error - backend crashed
```

JWT Flow

- 1. User POSTs credentials to /login
- 2. Server validates, returns access + refresh token
- 3. Client stores tokens (localStorage or cookie)
- 4. Client sends: Authorization: Bearer
- 5. Server validates signature, no DB lookup needed
- 6. Access token expires → use refresh token to get new one

CORS — Cross Origin Resource Sharing

- Browser blocks requests from different origin
- Django: django-cors-headers, CORS_ALLOW_ALL_ORIGINS
- Next.js calls Django API — CORS must be configured
- Common issue: forgot CORS → 'Access-Control-Allow-Origin' error

8. YOUR PROJECTS — ANSWERS READY

RentEase — The One They'll Ask Most

What is RentEase?

"RentEase is a full-stack SaaS platform I built to digitize Pakistan's informal rental market. Landlords and tenants currently manage agreements on WhatsApp or paper — RentEase replaces that with digital agreements, payment tracking, and notices."

Tech stack and why?

"Next.js for frontend — server-side rendering for better performance and SEO. Django REST Framework for backend — Python's ecosystem is great for rapid API development with clean ORM. PostgreSQL for relational data — landlord-tenant-property-payment relationships need proper foreign keys. FastAPI microservice for PDF generation — kept it separate so the main API stays clean. JWT for stateless authentication."

Biggest challenge?

"Role-based access control — landlords and tenants see completely different data. I implemented custom permission classes in Django and role-checked every endpoint. On the frontend I used protected routes and stored the role in the JWT payload."

If they ask about Supabase in FYP (HealthMate):

"In HealthMate I used Supabase for real-time health data storage and auth. Supabase's PostgreSQL backend was a natural choice — it gave us real-time subscriptions for live health metric updates and built-in auth so we didn't need to build a separate auth service."

Q: Walk me through your RentEase architecture.
A: Next.js frontend on port 3000, Django REST backend on port 8000, FastAPI PDF microservice on port 8001, PostgreSQL database. Frontend calls Django API with JWT token in header. Django validates token, queries PostgreSQL via ORM, returns JSON. For PDF generation, Django calls FastAPI which returns PDF bytes. Role-based — landlords manage properties, tenants view agreements and pay.

Q: How did you handle authentication in RentEase?
A: Used SimpleJWT library in Django. On login, server returns access token (1 day) and refresh token (7 days). Frontend stores in localStorage, sends as Authorization: Bearer header. Django JWTAuthentication middleware validates every request. Custom IsLandlord permission class restricts property CRUD to landlords only.

Q: What would you improve in RentEase?
A: Add Redis caching for frequently read data like property listings. Add Celery for background tasks — rent reminder emails, lease expiry notifications. Move to Supabase for real-time payment status updates without polling. Add proper test coverage with pytest.

9. SALARY & QUESTIONS TO ASK THEM

When asked salary expectation:

"I'm looking for 80,000 to 90,000 PKR. I'm a fresh graduate and I understand this role typically requires 1-2 years experience — I'm confident my project portfolio demonstrates that level of practical skill and I'm committed to proving that quickly."

Why this company / role?

"Your stack — React, Next.js, Django, Supabase — maps exactly to what I've been building. I want to work in a team environment where I can apply these skills on real products and grow quickly. The 2PM-11PM timing works well for me as I'm used to working in focused evening blocks."

Questions YOU ask them — ask 2-3:

Question	Why it's smart
What does a typical day look like for a developer here?	Shows you want to understand the role, not just get hired
What's the biggest technical challenge the team is working on right now?	Shows genuine technical curiosity
How does the team handle code reviews?	Shows you care about code quality
Is there opportunity to move to a full-time role after the internship period?	Important — clarify future
What's the onboarding process like for new developers?	Shows you're thinking about contribution

10. LAST 10 MINS BEFORE INTERVIEW

"Supabase is PostgreSQL with a REST API layer, real-time subscriptions, and built-in auth — I used it in HealthMate FYP for real-time health data and auth."

"RentEase: Next.js frontend, Django REST backend, PostgreSQL database, FastAPI microservice for PDF generation, JWT authentication, role-based access control."

"useEffect with [] runs once after mount. Without [] runs after every render. With [dep] runs when dep changes."

"SSR: server generates page per request — fresh data. SSG: pre-built at build time — fastest, CDN-served."

"JWT: user logs in, server returns access + refresh token. Client sends Authorization: Bearer token. Stateless — no session in DB."

"Supabase RLS: Row Level Security — database-level policies restricting which rows each user can access. auth.uid() matches logged-in user."

"select_related = SQL JOIN for ForeignKey. prefetch_related = separate query for ManyToMany. Both prevent N+1 problem."

"CORS must be configured in Django when Next.js calls the API from a different port — django-cors-headers."

"My salary expectation is 80,000 to 90,000 PKR as a fresh graduate."